Smart Launcher

Sistema de siembra aérea de propágulos de mangle

Guía de Inicio para Nuevos Integrantes

Versión del documento: 1.0

Fecha: 13 de agosto de 2026

Repositorio: `Smart-Mangrove` — rama `sim-mode-and-rc-trigger`

> **Antes de conectar cualquier cosa:** lea completa la Sección 1 (Seguridad). Este documento asume que usted no conoce el proyecto, pero sí que trabajará con cuidado. El equipo puede dañarse de forma irreversible si se omiten los pasos de seguridad.

Guía de Inicio

Este documento está dirigido a alguien que se incorpora al proyecto sin conocimiento previo. Para el historial completo y las razones detrás de cada decisión técnica, consulte `PROJECT_HANDOFF.md`; ese documento es denso y está escrito para alguien ya familiarizado. **Lea primero esta guía.**

Contenido

1. Seguridad — leer antes de tocar nada
2. Qué es este proyecto
3. El hardware
4. Cableado existente
5. Preparación del computador
6. Identificar el puerto COM
7. Cargar el firmware
8. Uso del panel de control
9. Estado real del proyecto
10. Resolución de problemas
11. Qué hacer a continuación
12. Reglas y control de versiones
13. Glosario

1. Seguridad — leer antes de tocar nada

Estas advertencias no son formalidades. Cada una corresponde a un incidente real ocurrido en este proyecto o a un riesgo comprobado.

1.1 El lanzador es una máquina que se mueve

> **El firmware está actualmente en MODO SIMULACIÓN (SIM).** Los servomotores están desconectados por software y, en su lugar, se enciende un LED. Ese es el estado seguro. **No lo cambie** salvo que haya alguien con experiencia acompañándolo.

- **Mantenga desconectada la alimentación de los servos (el UBEC),** incluso en modo simulación.
- La tapa de las semillas se desplaza **en un solo sentido**. Si avanza, no regresa sin desarmar el mecanismo. Un error aquí cuesta trabajo real.
- Ya ocurrió un incidente en el que un servo giró de forma continua y no se detuvo; hubo que cortar la alimentación físicamente. Por eso se prueba con LEDs.

1.2 Reglas eléctricas

- El **ESP32 funciona a 3.3 V y NO tolera 5 V**. Conectar una señal de 5 V directamente a cualquiera de sus pines puede destruirlo.
- El Arduino Mega funciona a **5 V**. Toda señal que vaya del Mega al ESP32 requiere un divisor de tensión (Sección 4).
- **La tierra común (GND) es obligatoria.** Dos placas que intercambian una señal deben tener sus pines GND unidos. Sin tierra común la señal no funciona, y la falla parece un error de software.
- Nunca conecte dos salidas (dos pines TX) entre sí.

1.3 Reglas con la aeronave

> **Hélices desmontadas siempre** que el M600 esté energizado en mesa de trabajo. Sin excepciones. No vuele nada: ningún elemento de este proyecto está validado para vuelo.

2. Qué es este proyecto

Un dron siembra propágulos de mangle para reforestar manglares.

El **Smart Launcher** es un módulo que se monta bajo el dron. Almacena semillas detrás de 5 compuertas deslizantes por módulo, y existen 2 módulos (A y B) montados simétricamente para mantener el equilibrio de la aeronave.

> **Idea central:** el lanzador realiza toda su lógica de forma autónoma. Tiene su propio microcontrolador, su propio GPS y su propia configuración. El dron únicamente vuela y suministra energía. Gracias a esto, el mismo lanzador funciona en un DJI M600 o en un dron de construcción propia cambiando un archivo de configuración, no el programa.

Cuando el lanzador determina que llegó a un punto de siembra, acciona un servomotor que desplaza la tapa el ancho de una compuerta (35 mm) y la semilla cae por gravedad. Primero dispara el Módulo A, espera confirmación, y luego el Módulo B. Ese orden mantiene equilibrada la aeronave.

2.1 Cómo sabe dónde está

Fuente	Descripción
OSDK	Lee la posición directamente del controlador de vuelo del DJI M600 por cable serie, 50 veces por segundo. Es la mejor opción y ya funciona.
GNSS	Módulo GPS propio (u-blox M8N). Respaldo automático.
Simulada	Posición ficticia introducida manualmente, para pruebas en interiores.

2.2 Cómo decide sembrar

1. **Automático:** se guarda una lista de coordenadas. Cuando el lanzador entra en el radio configurado de una de ellas, siembra.
2. **Disparo del piloto:** el piloto acciona un interruptor del radiocontrol. Está implementado pero *nunca se ha probado de extremo a extremo*.

3. El hardware

Elemento	Función	Ubicación
ESP32 WROVER-E	Cerebro del lanzador. Ejecuta todo.	Placa principal
Arduino Mega 2560	Herramienta de prueba: genera una señal de radiocontrol simulada.	Solo banco, nunca vuela
Adaptador FTDI FT232	Cable USB–serie para escuchar al DJI desde el PC.	Solo banco
DJI M600 / A3 Pro	Aeronave y su controlador de vuelo.	—
Servos FS5106R	Motores que desplazan las tapas.	Actualmente desconectados

3.1 Asignación de pines del ESP32

Pin	Uso
GPIO 12	Señal del servo A
GPIO 13	Señal del servo B
GPIO 32	Fin de carrera A
GPIO 33	Fin de carrera B
GPIO 22	LED de estado (parpadea un patrón según el estado)
GPIO 27	LED amarillo de «en espera»
GPIO 14	LED de aviso de siembra
GPIO 35	Entrada del disparo del piloto (pin de solo entrada)
GPIO 25	Entrada serie del OSDK (DJI)
GPIO 26	Salida serie del OSDK (cableada, nunca utilizada)
GPIO 4 / 5	Módulo GPS

> PINES QUE NUNCA DEBEN USARSE
>
> **GPIO 2** — la placa no arranca si se controla este pin. Costó una sesión completa de diagnóstico.
> **GPIO 9 y 10** — están conectados físicamente al chip de memoria interno del módulo. Costó otra sesión.
> **GPIO 16 y 17** — mismo problema, también conectados a la memoria. Se detectó antes de causar daño.
>
> Si necesita un pin libre, use **18, 19, 21 o 23**. La lista autorizada, con las razones, está en `Servo_Web_Calibrator_WROVER/Pins.h`. Léala antes de cablear.

4. Cableado existente

4.1 Los tres LEDs

Cada LED se conecta así: `pin del ESP32 → resistencia de 330 Ω → pata larga del LED (+) → pata corta (−) → GND`.

LED	Pin	Qué indica
Estado	22	El patrón de parpadeo indica el estado del sistema
Amarillo «espera»	27	**Encendido fijo** = armado y esperando un punto de siembra
Aviso de siembra	14	Se enciende cuando se ejecuta una siembra

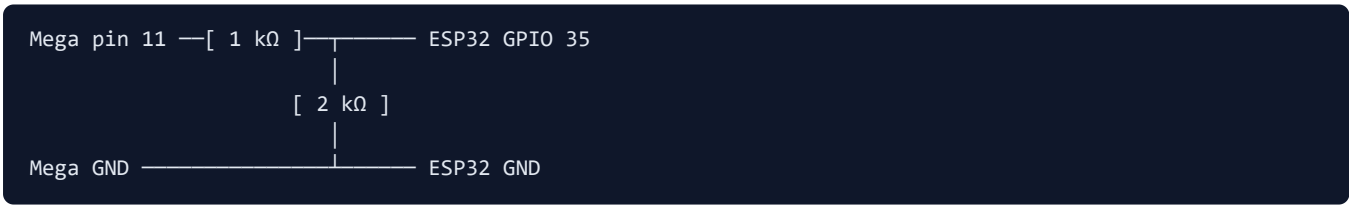
4.2 Cable OSDK del DJI (datos de posición)

```
Puerto «API» del A3  TX   →  [330 Ω]  →  ESP32 GPIO 25
Puerto «API» del A3  GND  →              ESP32 GND      ← obligatorio
Puerto «API» del A3  RX / VCC            NO CONECTAR
```

Ambos lados trabajan a 3.3 V, por lo que no se requiere conversor. **Este enlace es de solo recepción**: el lanzador escucha, nunca envía órdenes a la aeronave. Es una regla de diseño deliberada, no una limitación.

4.3 Señal de prueba desde el Arduino Mega

El Mega trabaja a 5 V, por lo que **requiere divisor de tensión**:



> **Use 1 kΩ / 2 kΩ. NO use 10 kΩ / 20 kΩ.** El GPIO 35 ya tiene una resistencia de 10 kΩ, y la combinación reduce la tensión por debajo del umbral lógico. El síntoma es idéntico al de un cable roto.

5. Preparación del computador

5.1 Instalar el Arduino IDE

Versión 2.x. En el equipo del proyecto ya está instalado en `C:\Program Files\Arduino IDE\`.

5.2 Soporte para placas ESP32

En `Archivo → Preferencias → Gestor de URLs adicionales`, agregue:

```
https://espressif.github.io/arduino-esp32/package_esp32_index.json
```

Luego `Herramientas → Placa → Gestor de tarjetas`, busque **esp32** e instálelo. El equipo del proyecto usa la **versión 3.3.8**; utilice esa si es posible.

5.3 Configuración de la placa

Opción	Valor
Placa	**ESP32 Wrover Module**
Upload Speed	921600
Flash Frequency	80 MHz
Partition Scheme	Default 4MB with spiffs

> **«ESP32 Wrover Module» no es opcional.** Seleccionar «ESP32 Dev Module» configura mal el chip de memoria y la placa se reinicia continuamente. Ya ocurrió una vez.

5.4 Activar los avisos del compilador

`Archivo → Preferencias → Advertencias del compilador → Por defecto` (o superior).

El código contiene un aviso deliberado que aparece al compilar una versión **de vuelo**, recordando que hay tres errores conocidos sin resolver. Con la opción «Ninguna» ese aviso nunca se vería.

6. Identificar el puerto COM

> **El número de puerto cambia.** Desconectar y volver a conectar puede mover el ESP32 de COM6 a COM11 o a cualquier otro. Nunca lo dé por supuesto.

1. Conecte la placa.
2. Arduino IDE → `Herramientas → Puerto`. Anote lo que aparece.
3. Desconecte la placa y vuelva a mirar. El que desapareció es el suyo.

Valores observados recientemente (*no confiar*): ESP32 en COM6, Arduino Mega en COM8, adaptador FTDI en COM11.

> **Solo un programa puede usar un puerto a la vez.** Si la carga falla con «Acceso denegado» o «puerto ocupado», casi con certeza el Monitor Serie sigue abierto. Ciérrelo y reintente. Esto nos ocurrió tres veces.

7. Cargar el firmware

7.1 Firmware principal (panel de control)

1. Abra `Servo_Web_Calibrator_WROVER/Servo_Web_Calibrator_WROVER.ino`
2. Verifique en `Pins.h` la línea `#define SIM_MODE` — **debe decir** `1`
3. Configure placa y puerto (Secciones 5 y 6)
4. Pulse la flecha → (Subir)
5. Espere `Hash of data verified` y `Hard resetting`

7.2 Verificar que funcionó

Abra el Monitor Serie a **115200 baudios** y pulse el botón **EN/RST** de la placa. Deben aparecer doce pasos numerados:

```
=== Smart Launcher Mission Dashboard ===
[1] Config (LittleFS + mission.json...)
[2] Logger
[3] State machine (LED GPIO22...)
[4] Position source (SIM — real OSDK/GNSS until /simpos injects)
[5] Actuators *** SIM MODE — announce LED GPIO14,
    servo GPIO12/13 NOT attached ***
[6] Drop logic engine
[6b] RC pilot trigger (GPIO35 ...)
[7] WiFi AP
[8] IP: 192.168.4.1
[9] HTTP server
[10] BOOT -> IDLE
[11] Ready — connect to SmartLauncher -> http://192.168.4.1
[12] setup() complete — loop starting
```

> La línea `[5]` que dice **«servo GPIO12/13 NOT attached»** es la prueba de que el modo simulación está activo y nada puede moverse físicamente. Búsquela siempre.

> Si aparece `rst:0x8 (TG1WDT_SYS_RESET)` repetidamente, la placa está en ciclo de reinicio. Deténgase y consulte la Sección 10.

7.3 Otros programas (herramientas de prueba)

Archivo	Dónde se carga
`RC_Input_Calibration_Tool.ino`	ESP32 — mide una señal de radiocontrol
`RC_Pulse_Generator_Mega2560.ino`	Arduino Mega — genera una señal de prueba
`tools/osdk_sniff.py`	PC (Python + `pip install pyserial`)

8. Uso del panel de control

1. Energice el ESP32.
2. Conéctese a la red Wi-Fi `SmartLauncher` (sin contraseña).
3. Abra `http://192.168.4.1`

Perderá el acceso a internet mientras esté conectado: es una red independiente generada por la placa.

> Debe aparecer un recuadro naranja que dice **SIM MODE — NO SERVO OUTPUT**. Si ese aviso no está, el firmware puede accionar motores reales. Deténgase y verifique `SIM_MODE`.

8.1 Las tres pestañas

Pestaña	Contenido
Control	Armar/desarmar, siembras manuales, parada de emergencia y registro de vuelo. *ARM* requiere dos toques. La **PARADA DE EMERGENCIA** es la barra roja inferior: un toque, siempre disponible, sin confirmación. *worst loop* indica el ciclo más lento del software; unos pocos ms es saludable.
Waypoints	Planificación de la misión. **ADD WAYPOINT HERE** captura la posición actual con un solo toque; se rechaza si no hay señal GPS. El radio define a qué distancia se activa la siembra: **use 10 m para pruebas**. Los cambios solo se guardan al pulsar *Save*.
Calibration	Ajuste de servos y del disparo por radiocontrol. **No use los controles de servo** sin indicación expresa.

9. Estado real del proyecto

9.1 Comprobado en hardware real

- El panel de control funciona; las tres pestañas se muestran correctamente.
- Lectura de posición desde el DJI M600 a ~47 actualizaciones por segundo, verificada en exteriores con señal GPS real.
- La lógica de siembra se activa correctamente al alcanzar un punto, comprobado caminando hasta él y observando el LED.
- El registro de vuelo guarda eventos en un archivo de la placa.
- La herramienta de lectura de radiocontrol funciona contra un generador de señal.

9.2 Implementado pero NO comprobado

> - El disparo por radiocontrol del piloto nunca se ha probado de extremo a extremo.
> - **Los servos nunca han sido accionados por este firmware. Ni una sola vez.**
> - El hardware del Módulo B todavía no existe.
> - Nada ha volado.

9.3 Tres errores conocidos permanecen en el código A PROPÓSITO

Fueron detectados leyendo el código pero nunca confirmados en ejecución. Se dejaron deliberadamente para que la próxima prueba los confirme o los descarte, en lugar de que alguien «corrija» un problema que quizá no exista.

1. **La parada de emergencia podría accionar el Módulo B.** Al pulsar parada durante una secuencia automática, el módulo B podría iniciarse igualmente.
2. **La parada de emergencia podría no desarmar.** Podría dejar el sistema en estado ARMADO.
3. **10 puntos frente a 5 compuertas.** Si la misión tiene más puntos que compuertas, los sobrantes se registran como exitosos sin que ocurra nada.

> **No conecte los servos hasta que estos tres puntos estén resueltos.** La pestaña *Waypoints* advierte automáticamente sobre el punto 3.

10. Resolución de problemas

Síntoma	Causa y solución
La carga falla: «Acceso denegado» / puerto ocupado	El Monitor Serie está abierto. Ciérrelo.
La carga falla: puerto no encontrado	El número de puerto cambió. Revise `Herramientas → Puerto`.
`rst:0x8` en bucle	Placa incorrecta seleccionada, o algo conectado a GPIO 2 / 9 / 10 / 16 / 17.
El monitor muestra caracteres extraños	Velocidad incorrecta. Configure 115200 baudios.
El panel no carga	¿Está conectado a la red `SmartLauncher`? La dirección es `192.168.4.1`, sin https.
«no fix» en exteriores	Revise las líneas `[OSDK]` del monitor serie. `frames=0` indica cable incorrecto o A3 apagado. Si hay frames pero `health` menor a 3, la aeronave aún no fija satélites: espere un minuto.
No se ejecuta la siembra	¿Está ARMADO? ¿Hay un punto guardado? ¿El radio es suficiente? ¿El punto está realmente cerca?
No aparecen siembras en el registro	Active «Events only». La telemetría escribe una línea por segundo y las oculta.
Todo responde con lentitud	Revise *worst loop*. Si es alto, probablemente el registro esté lleno: bórrelo desde la pestaña Control.

10.1 Lectura de los LEDs

Amarillo (27)	Aviso (14)	Significado
parpadeo lento	apagado	Inactivo, sin armar
fijo	apagado	Armado, esperando
apagado	**encendido ~0.6 s**	Módulo A ejecutando
apagado	1 destello corto	Compuerta del Módulo A confirmada
apagado	**encendido ~0.35 s**	Módulo B ejecutando
apagado	1 destello largo	Compuerta del Módulo B confirmada
parpadeo rápido	apagado	Fallo

Destellos cortos = Módulo A; destellos largos = Módulo B. Así se distinguen con un solo LED.

11. Qué hacer a continuación

En este orden. No salte pasos.

1. **Verifique el registro de vuelo** y bórrelo si está lleno (pestaña Control → Clear, dos toques).
2. **Repita la prueba caminando** y observe si persiste la lentitud al sembrar. Vigile *worst loop*.
3. **Ejecute las pruebas de parada de emergencia.** Arme, provoque una siembra y pulse PARADA DE EMERGENCIA mientras el LED de aviso está encendido.
   - Si el LED vuelve a encenderse → el error 1 es real. Anótelo.
   - Si el estado permanece ARMADO → el error 2 es real. Anótelo.
4. **Registre los resultados en** `SIM_MODE_VALIDATION_PLAN.md`, que contiene una tabla con celdas en blanco esperando exactamente estos datos.

> `SIM_MODE_VALIDATION_PLAN.md` es el guion de pruebas: 20 pasos numerados con un criterio de aprobación cada uno. **En algunos pasos, que la siembra se ejecute significa FALLO** — son los que verifican que algo *no* debe ocurrir. Lea el criterio, no lo suponga.

11.1 Observación pendiente sobre el tiempo de respuesta

Se identificó y corrigió una causa importante de lentitud: un mensaje de diagnóstico que se imprimía por el puerto serie en cada ciclo del programa mientras el sistema estaba armado. Sin embargo, el indicador *worst loop* todavía muestra picos de 60–200 ms cuyo origen no está plenamente aislado; la evidencia apunta al propio volumen de mensajes por puerto serie. Es un punto abierto, no resuelto.

Además, el punto de siembra guardado actualmente tiene un **radio de 1.0 m**. La posición GPS fluctúa entre 0.5 y 1 m incluso estando quieto, por lo que ese radio es demasiado estrecho y puede explicar buena parte de la lentitud percibida. **Use 10 m para las pruebas.**

12. Reglas y control de versiones

Haga	No haga
Mantener `SIM_MODE 1` Mantener el UBEC de los servos desconectado Anotar lo observado, incluso cuando funciona Consultar antes de cambiar asignaciones de pines	No conectar los servos No volar nada No corregir los tres errores conocidos antes de probarlos No usar GPIO 2, 9, 10, 16, 17 No fusionar la rama con `main`

El trabajo está en la rama `sim-mode-and-rc-trigger`, publicada en GitHub. Deliberadamente **no** está fusionada con `main` por los tres errores conocidos.

```
git status                  # ¿qué he modificado?
git add -A                  # preparar todos los cambios
git commit -m "mensaje"     # guardar un punto de control
git push                    # enviar a GitHub
```

Haga *commit* cada vez que algo funcione. Es gratis y permite volver atrás siempre.

13. Glosario

Término	Significado
ESP32	El pequeño computador que ejecuta el lanzador
Flashear / cargar	Copiar el programa a una placa
Monitor Serie	Ventana que muestra el texto que imprime la placa. Herramienta esencial de diagnóstico.
Baudios	Velocidad de la conexión serie. Aquí, 115200.
Puerto COM	Nombre que Windows da a una placa conectada (COM6, COM8…)
GPIO	Pin numerado del ESP32
PWM	Pulso cuyo *ancho* transmite un valor. Se usa en servos y radiocontrol.
UART / serie	Dos cables que envían datos bit a bit
OSDK	Onboard SDK de DJI: cómo leemos la posición del M600
LittleFS	Sistema de archivos del ESP32 donde viven `mission.json` y el registro
mission.json	Archivo de configuración: puntos, valores de servo, ajustes de radiocontrol
JSONL	Formato del registro: un objeto JSON por línea
Waypoint	Coordenada GPS donde debe caer una semilla
Armar	Poner el sistema en un estado donde se le permite sembrar
UBEC	Pequeña fuente que alimenta los servos a 5 V
Haversine	Fórmula de distancia entre dos coordenadas GPS
Modo SIM	Modo seguro de prueba: LED en lugar de servos

14. Mapa de documentos

Archivo	Contenido
`START_HERE.md`	Versión en inglés de esta guía
`SIM_MODE_VALIDATION_PLAN.md`	Guion de pruebas. **Es el trabajo siguiente.**
`PROJECT_HANDOFF.md`	Historial completo y justificaciones. Denso pero autorizado.
`DESIGN_RATIONALE.md`	Por qué el lanzador tiene su propio microcontrolador
`SMART_LAUNCHER_COMPLETE_GUIDE.md`	Cableado, lista de materiales, construcción mecánica
`Servo_Web_Calibrator_WROVER/`	**Firmware principal**
`Servo_Web_Calibrator_WROVER/Pins.h`	Asignación de pines — la referencia autorizada
`tools/osdk_sniff.py`	Herramienta de PC: decodifica la telemetría DJI

> **Si se queda atascado:** anote qué hizo, qué esperaba, qué ocurrió y cualquier mensaje del Monitor Serie. Ese hábito ha resuelto la mayoría de los problemas difíciles de este proyecto: varios errores se encontraron porque alguien notó que faltaba una línea que *debía* estar.

Nada aquí es tan frágil como para romperse trabajando con cuidado. Lo único verdaderamente irreversible es accionar los servos, y por eso están desconectados.
